Shloka Kulkarni

AI Engineer and full-stack developer

kulkarni.shloka03@gmail.com +91 8007733622 Pune, Maharashtra, India

shlokakulkarni.vercel.app linkedin.com/in/shlokakulkarni github.com/shKul03

Summary

AI Engineer and full-stack developer with production experience at CrowdStrike, Kashnate Solutions, and Technossus. Specialises in RAG pipelines, intelligent document processing, voice bot orchestration, and LLM-powered backend systems. Comfortable across Python, TypeScript, C#, and cloud-native infrastructure (Docker, Kubernetes, Azure, AWS). Currently part of the AI Studio team at Technossus — researching, experimenting, and shipping AI-first products. CGPA 9.41, two-time semester rank-first at MIT-WPU.

Professional Experience

AI Engineer 01/2026 – Present

Technossus — AI Studio Pune, India

- Part of the AI Studio team: a dedicated unit focused on researching, experimenting, and shipping AI-first products and internal tooling.
- Architected TechnoAI — a zero-storage RAG system that crawls any website via Playwright (sitemap + fallback), stores vectors in PostgreSQL/pgvector, and answers domain-scoped queries with session memory; new domains deployable in one day.
- Built KnowledgeEngine — an enterprise RAG backend with custom embeddings, LLM generation, a re-ranker layer, and full document lifecycle management with test coverage for parser, classifier, and vector store modules.
- Designed OnBoardIQ — a local-first AI Background Verification pipeline: classifies KYC/HR documents (Aadhaar, PAN, Passport, Degrees), deduplicates via byte -> perceptual -> semantic hashing, organises per-candidate file trees, and generates Excel audit reports. Feature-flag architecture supports PoC -> Enhanced -> Enterprise rollout without code rewrites.
- Delivered SmartRevenueCollector — end-to-end AI debt-collection automation with ML defaulter scoring, LLM-generated personalised messages, a prioritisation queue, and a React/TypeScript dashboard deployed to GitHub Pages.
- Contributed to an in-house Voice Bot PoC: built the Python/FastAPI STT/TTS microservice and co-developed the C#/.NET orchestrator backend using Clean Architecture (API -> Application -> Domain -> Infrastructure).
- Built DemonotesAI — a modular agentic notes assistant with separate agents, services, clients, models, and prompts layers; reusable scaffold for AI-augmented productivity tooling.

Backend Developer 07/2025 – 12/2025

Kashnate Solutions Pune, India

- Developed and maintained Python-based backend services and REST APIs for client-facing web applications.

- Built AI-assisted frontend features using Node.js, integrating LLM capabilities into the user-facing layer.
- Collaborated closely with the frontend pipeline to ensure seamless API contracts and end-to-end feature delivery.

Malware Research Engineer 06/2024 – 06/2025
CrowdStrike Remote / India
- Designed, developed, and deployed production-grade services using Python, TypeScript, and React in a security-critical environment.
- Built a fault-tolerant scheduling microservice that reduced processing delays by 30% through optimised database queries and backpressure handling.
- Applied containerisation and orchestration using Docker, Kubernetes, and Rancher for scalable, resilient deployments.
- Utilised Amazon OpenSearch, Elasticsearch, and Amazon S3 as backend data stores; debugged and resolved production incidents.
- Collaborated on API integrations and end-to-end software solutions across cross-functional engineering teams.

Backend Software Engineer 01/2024 – 06/2024
Comarete Technologies Pune, India
- Developed Python-based microservices and REST APIs to build scalable backend solutions.
- Enhanced application performance by 20% through MySQL and MongoDB query optimisation.
- Conducted unit testing with pytest; collaborated with front-end team to enable HTML/CSS/JS features.

Notable Projects

TechnoAI
Python · FastAPI · pgvector · Playwright · Ollama · Docker
- Conversational RAG AI chatbot that can be deployed on any website in under a day — scrapes the target site live using Playwright (sitemap-first with fallback crawl), chunks and embeds content into PostgreSQL/pgvector using nomic-embed-text, and answers user queries conversationally with session memory.
- Zero document storage architecture: no manual uploads, no maintenance overhead. Ingest is triggered via a single API call; re-ingestion clears and rebuilds vectors automatically.
- Configurable via environment variables — WEBSITE_URL, chunk size, overlap, retrieval top-k, and minimum cosine similarity threshold. Fully containerised with Docker Compose (FastAPI + PostgreSQL/pgvector).
- Cosine similarity retrieval with 1 - distance scoring (1.0 = identical, 0.0 = orthogonal); session memory maintained per UUID across multi-turn conversations.

Knowledge Engine
Python · FastAPI · pgvector · Re-ranker · Custom Embedder
- Enterprise-grade RAG pipeline: document ingestion → chunking → custom embedding → pgvector storage → cosine similarity retrieval → cross-encoder re-ranking → LLM response generation.
- Full document lifecycle management — ingest, update, delete, and re-embed individual documents without full pipeline re-runs. Includes seed scripts for initial data population.
- Modular architecture: vector store, embedder, re-ranker, and LLM generation are independently swappable components. Test suite covers parser, classifier, and vector store modules.

- FastAPI backend with documented endpoints for ingestion, querying, document management, and health checks. Designed for multi-tenant enterprise deployment.

OnBoardIQ

Python · Ollama · Tesseract OCR · Streamlit · Google Drive (Phase 2)

- 3-phase BGV document processing pipeline: Phase 1 (PoC) — local classification and deduplication; Phase 2 (Enhanced) — Google Drive integration and advanced OCR; Phase 3 (Enterprise) — workflow controls, approval queues, and audit compliance.
- Classifies 10+ document types (Aadhaar, PAN, Passport, Degree, Resume, Offer Letter, etc.) using a hybrid rule-based + LLM classifier; falls back to LLM only for ambiguous cases, keeping processing fast and cost-efficient.
- 3-layer deduplication: byte hash (exact duplicates) → perceptual hash (visually similar scans) → semantic text similarity (same document, different format). Catches duplicates that single-method approaches miss.
- Organises processed documents into per-candidate folder trees and generates Excel audit reports with classification confidence scores, duplicate flags, and processing timestamps.
- Feature-flag architecture allows each phase to be enabled independently in production — teams can ship Phase 1 immediately and activate Phase 2/3 features via config without code changes.

Smart Revenue Collector

Python · FastAPI · SQLite · React · TypeScript · Tailwind · Vite

- End-to-end AI debt-collection automation: data ingestor → ML defaulter scoring algorithm → priority queue → LLM-generated personalised outreach messages → dispatch module → response triage.
- ML scoring model ranks defaulters by likelihood-to-pay and outstanding amount; priority queue ensures highest-impact cases are contacted first. Penalty/incentive engine adjusts messaging tone based on account history.
- LLM generates contextually personalised messages per defaulter — name, amount, history, and tone are dynamically injected. Message variants tested for response rate optimisation.
- React + TypeScript frontend with live analytics dashboard: defaulter distribution, contact success rates, collection pipeline status, and per-account interaction history. Deployed to GitHub Pages.
- Customer-facing delivery: presented solution architecture and live demo to stakeholders; incorporated feedback directly into feature prioritisation and UI design decisions.

Bill Sage AI

Python · OCR · FastAPI · Streamlit

- Asynchronous OCR pipeline for intelligent bill classification — accepts image and PDF uploads, extracts structured data, and classifies bills by type (utility, grocery, medical, etc.).
- Async upload handling with concurrent processing support; modular app/dashboard separation. Streamlit analytics dashboard displays classification results, confidence scores, and extraction accuracy metrics.
- Tested with async upload simulation suite to verify pipeline reliability under concurrent load.

SentiCure

AI · WhatsApp Integration · NLP · Python · Queue Management

- AI-powered patient appointment system delivered entirely over WhatsApp — no app installation or website visit required. Patients interact with an AI that collects symptoms, suggests the appropriate specialist, checks real-time doctor availability, and manages appointment queues.
- Integrated AI model handles natural, human-like conversation flow: symptom triage → specialist recommendation → slot booking → live queue position updates, all within a single WhatsApp thread.
- Presales & client-facing: drove client discovery meetings, delivered end-to-end technical walkthroughs and live demos to hospital stakeholders, contributed to the formal sales pitch and solution documentation.

- Made direct feature additions to the AI implementation including conversation flow improvements and queue update notification logic.

Technossus Website V2
TypeScript · React · Tailwind CSS · Figma · Design Tokens · Vite
- Converted Figma design files directly to production code — established a complete design system from typography, colour themes, and spacing rules through to interactive React components.
- Design tokens in W3C format with CSS variable output; custom Tailwind preset for consistent utility usage across the codebase. Components include Tag, Stats Card, Testimonial, SearchBar, and navigation elements.
- Used the design system to build full website pages from Figma references, and later created entirely new pages by providing only content — demonstrating the system's completeness and developer ergonomics.
- Deployed live as a reference implementation; serves as the single source of truth for all Technossus frontend development.

VoiceBot
Python · FastAPI · C# · .NET · Clean Architecture
- Multi-service voice assistant built as an internal PoC at Technossus AI Studio: Python/FastAPI STT/TTS microservice handles speech-to-text and text-to-speech as standalone REST endpoints.
- C#/.NET orchestrator backend routes voice turn-by-turn interactions across STT → LLM → TTS services. Implements phone-number capture flow and session state management.
- Clean Architecture layering: API → Application → Domain → Infrastructure with dedicated test project. Designed for extensibility — new voice providers or LLM backends can be swapped at the infrastructure layer.

IoT & AI Entertainment Robot
Python · Raspberry Pi 3B+ · OpenCV · NLP
- Robot with facial recognition, obstacle detection, and NLP. First Runner-Up at college IoT competition.

🎓 Education

B.Tech in Computer Science Engineering 2021 – 2025
MIT World Peace University Pune, India
- CGPA: 9.41
- Ranked First in two semesters with a perfect 10.0 GPA
- Awarded Merit-based Scholarship in second year for academic excellence

Skills

Languages — Python, TypeScript, JavaScript, C#, Java, C++, SQL

AI / ML — RAG pipelines, pgvector, Ollama, LLMs, LangGraph, OCR (Tesseract, pypdf), NLP, CNNs, RNNs, scikit-learn

Backend — FastAPI, .NET / ASP.NET Core, Node.js, Express.js, REST APIs, WebSockets

Frontend — React, TypeScript, Vite, Tailwind CSS, ShadCN UI, HTML/CSS

Databases — PostgreSQL + pgvector, MySQL, MongoDB, SQLite, Amazon OpenSearch, Elasticsearch, Redis, S3

DevOps / Cloud — Docker, Kubernetes, Rancher, Azure DevOps / Pipelines, AWS, GitHub Actions

Architecture — Microservices, Clean Architecture (DDD), event-driven design, feature-flag gating, Agile/Scrum

Achievements

First Runner-Up — IoT Competition
Raspberry Pi AI Robot with facial recognition and NLP

First Runner-Up — Texephyr Coding Contest

Ranked First in two semesters with perfect 10.0 GPA
MIT World Peace University

Merit-based Scholarship
B.Tech Year 2, MIT World Peace University